CS5702 Week1 Lab Worksheet

Martin Shepperd

10/09/2026

This Worksheet is organised as an RMarkdown document. You can **read** it. You can **run** the embedded R and you can **add** your own R. We suggest you save it as another file so, if necessary, you can revert to the original.

Whenever you click **Knit** in the RStudio menu it will render the markdown into nicely formatted html (or pdf or even docx) which you can examine in the Viewing Window in RStudio or via any web browser. You may find this easier to read, however, you must edit the .rmd file, i.e., the RMarkdown in the Edit Pane if you want to make any changes. You are encouraged to *explore* and *experiment.* Change something and see what happens!

Note, that many of the topics and challenges are the same or closely interwoven with CS5701 (QDA) so never be afraid to carry over lessons and ideas from one module to the other.

Pre-requisites

1. You should have completed the Week 0 "Getting Ready" and minimally must Install and test R and RStudio material.
2. You should have attended/read the Week 1 Lectures.

0. Worksheet Topics

1. Create, save and edit R Notebooks using RStudio
2. Exploring the use of R using R
3. Some Basic R practice

A. *Extension*: Appendix - notes on YAML

B. *Extension*: Appendix - notes on {ggplot2}

0.1 Worksheet Navigation: Choose your track before starting

Welcome! Depending on your prior R experience, follow the appropriate pathway below:

TRACK A: New to R / Refreshing Core Concepts
Focus: Work step by step through **Sections 1, 2 and 3**. **Goal:** Take your time with the RStudio mechanics, running RMarkdown code chunks, basic variable assignment and fundamental functions. Do not feel pressured to reach the extension tasks (today).

TRACK B: Experienced R Users (e.g., January Starters) Focus: Fast-track through **Sections 1 and 3** but work through all of Section 2 as it's based on an actual data analysis example. Target all

Extension Exercises (e.g., *Exercise 2.6* and *Appendices 1 and 2* for YAML and `{ggplot2}` visualisation work).
Goal: Focus on critical evaluation (e.g., *Exercises 2.1 & 2.3*) and converting base R graphics into something more professional with {ggplot2}. Over the course of the next couple of weeks we will all fully transition to {ggplot2}.

Note: Both tracks use this single `.Rmd` file. Track B extends the core tasks rather than skipping key learning objectives.

1. Lab: create, save and edit RMarkdown documents using RStudio

This worksheet is written using RMarkdown and saved as an RMD file.

This is an extremely convenient way to combine explanatory text written in a very simple markdown language and interactive R. We will use RMarkdown regularly for all non-trivial R code in both modules (CS5701 and CS5702), including for labs and also for your coursework (CS5801) and exam (CS5802). So it's important you become comfortable with using RMarkdown documents.

If you've not already viewed my video "Introducing RStudio" or you lack confidence in using this important tool I suggest you should check it here.

1. You need a YAML header (see this file for an example, or click here for more information).

2. You can enter plain text as-is. Any simple editor will do or write them directly in RStudio. *Avoid* word processors such as Word because the text is not stored as plain text and includes hidden non-printing characters which means you will get lots of truly horrible side effects!

3. You can use `#`, `##`, etc for different heading levels. Notice how RStudio automatically generates a nested Outline (navigation) Pane for you.

4. If you start a line with `1.` etc this generates an enumerated list (as per this example). There must be a space between the full stop and the text. If you just want bullets you can use `-` instead.

    - You can nest lists by indenting by 4 spaces.

5. You can insert a code chunk by clicking the green *Insert (Code Chunk)* button on the Edit Pane toolbar or by pressing *Cmd+Option+I*. Or you could, if you really want to do it yourself, type the three backticks followed by the language in braces, then the code and close the chunk with another three backticks.

6. You can also view and edit your RMarkdown in 'Visual' mode which shows the result of your formatting. It's your choice but I suggest you stick with 'Source' mode to start with until you are confident editing the RMarkdown.

There's also a very helpful guide on RMarkdown from RStudio here.

Below is an example of an R code chunk which can be executed from within the Rmd document. You can do this by clicking the green triangle to the top right of the chunk area within the document. If you wanted to run all the code up until, and including, this code chunk you can click the grey triangle with the green bar [1].

```
# This is an R code chunk
print("Hello world")
```

[1] In this case there is no previous code so it would make no difference but often RMarkdown contains multiple code chunks.

```
## [1] "Hello world"
```

Exercise 1.1: Edit the R code to output "Goodbye cruel world" or some other message. Re-run the code chunk.

Exercise 1.2: Save the edited RMarkdown document When you click the *Knit* button you can generate HTML which can be previewed in the Viewer pane - bottom right). The raw R markdown has a suffix .Rmd and the rendered HTML will have a suffix .html. Note that you can Knit to other formats if you prefer, such as pdf [2].

Check you can see the html version. The preview shows you a rendered HTML copy of the contents of the editor. The output from the R code chunks is whatever was generated when it was last run in the RStudio. You can also view the file in any web browser.

Exercise 1.3: Add a new 3rd level heading named "My RMarkdown Exercise" here. Add an anchor label `{#<anchor-name>}`. Now at another point in the document you can link to your anchor, as follows: `[text to display](#anchor-name)`. Lines 24-28 of this RMarkdown file are examples.

Exercise 1.4: Create a new RMarkdown file using the `File > New File > R Notebook` menu option. RStudio provides you with a basic template which you can modify as needed. Add a meaningful title and your name as an author in the YAML header. Knit the new file using the `Knit` pull down menu.

Click on my video here if you want further guidance on creating and manipulating RMarkdown documents.

Extension exercise 1.5: You can also create a special type of Rmd document called an R Notebook (analogous to a Jupityer notebook for python). The main syntactic difference is in the choice of output in the YAML header so you will have `output: html_notebook`. The advantage of notebook files is that code chunks can be run interactively rather than as a batch when rendering (knitting) in Rmd documents. To explore the differences further, see the tutorial from posit here.

To dig deep there's a nice blog of R Markdown Tips.

So now you can handle the RMarkdown basics. Let's move onto R.

2. Exploring the use of R using R

2.1 Demand for data science software skills

How useful are R skills and tools? We will use a recent analysis of US job adverts in 2025. Source: Lightcast external site job postings data for the US between January 1, 2025 and December 31, 2025. The percentage is the ratio of unique postings which mention the skill to all unique postings linked to the 0*NET-SOC occupation (categorised as data science).

The top 10 tools/technologies required by the employers are:

```
Python (66%)
SQL (51%)
R (34%)
Tableau (22%)
Microsoft Power BI (19%)
Amazon Web Services (17%)
Microsoft Azure (13%)
TensorFlow (11%)
PyTorch (10%)
SAS (9%)
```

[2]The way the file is rendered into pdf relies upon you having an installation of LaTeX on your machine. This tends to stress our lab machines - sorry! So if this doesn't work for you just focus on the html or reading the RMarkdown directly.

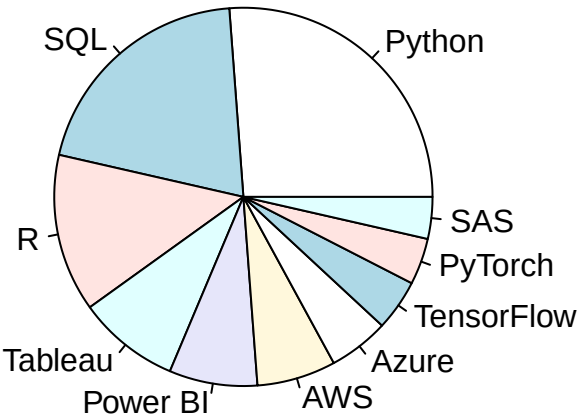
Exercise 2.1: It's important to look at the data critically, though we must accept nothing is certain or perfect! What questions or concerns might you have?

Since we're thinking about data and data analysis, it would be helpful to visualise the data. Therefore I have copied some of the data from their article (you can download a csv as well if you want) and plugged this into R (see below). Don't worry about the coding details for the present although be aware that for reasons of simplicity I'm just using Base R for the graphics (functions like `pie()` and `barplot()`). Later we will introduce the {ggplot2}[3] package which will enable almost complete control and flexibility in producing beautiful data visualisations. However, if you are already a confident R programmer feel free to jump ahead.

If you want to know more about installing or using packages in the R ecosystem see my video here.

```
# Reproduce the 2025 data from posted by 0*Net Online
# data on required tools for Data Scientists
# supplied by Lightcast.

skill <- c(
  "Python", "SQL", "R", "Tableau", "Power BI",
  "AWS", "Azure", "TensorFlow", "PyTorch", "SAS"
)
percent <- c(66, 51, 34, 22, 19, 17, 13, 11, 10, 9)
# Generate the pie chart
pie(percent, main="Pie chart of in demand data scientist tools", labels=skill)
```



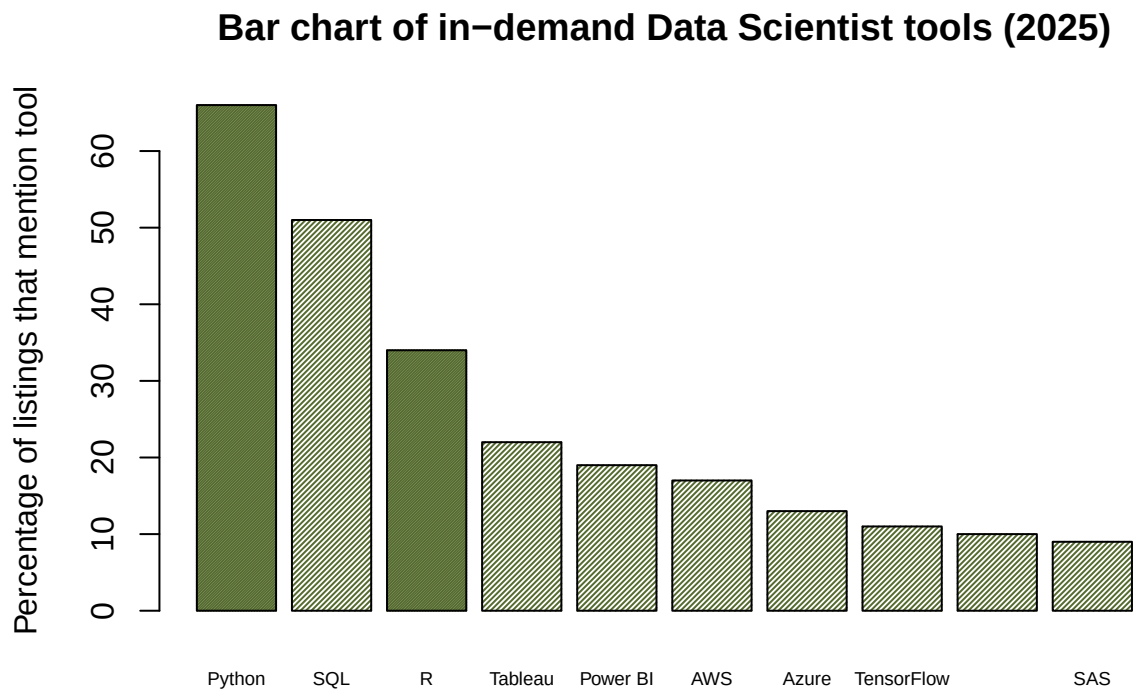
Exercise 2.2: Using the `clockwise` parameter change the order of display. Type `?pie` for help if required.

Exercise 2.3: If you look at the help for the `pie()` function you will read "Pie charts are a very bad way of displaying information." Why do you think this is so? [Hint: what might be the impact of multi-valued data, e.g., many adverts will ask for more than one skill? Also, how easy is it to read the values?]

[3] I, and many authors, use the convention of curly braces to denote an R package, so {ggplot2} means the package named ggplot2 which you could install with `install.packages("ggplot2")` and then read into memory with `library(ggplot2)`. If you're already an expert you can look at Appendix 2 to see how to do a pie chart with {ggplot2}, however, be warned it's a bit fiddly and also you're not really encouraged to make pie charts due to their many disadvantages. Barplots might be a better choice!

```r
# NB we have already reproduced the data: percent and skill come from the
# previous piechart chunk

# Generate a bar chart
barplot(percent,
main="Bar chart of in-demand Data Scientist tools (2025)",
sub = "Source: https://www.onetonline.org/link/demand/15-2051.00",
ylab="Percentage of listings that mention tool",
names.arg=skill,
cex.names = 0.6,       # shrinks text size so fits on x-axis
col="darkolivegreen",  # colour of the bars
                       # Hatches non-Python and R lighter
density=c(100, 50, 100, 50, 50, 50, 50, 50, 50, 50))
```



Exercise 2.4: Hopefully you consider this bar chart to be an improvement, but it's not perfect. Can you think of any further improvements or clarifications? (Hint: some of the bars are quite hard to read off the exact proportion.) Remember that if the goal of Data Science is to *communicate* findings, then how we present information is of vital importance. For this reason we will return to this topic in Week 4 to dig deeper on data visualisation.

Extension Exercise 2.5: The same article also gives the top 21 skills required and whether they are considered to be "hot technology", in an associated csv file. Can you think how you might integrate this additional information in an effective visualisation? Later in this worksheet we touch on how to read in files to save scraping and typing here.

Extension exercise 2.6: The above two plots are performed using base R (which is perfectly adequate for simple charts). A powerful and flexible alternative is the {ggplot2} package. How might you reproduce these charts using {ggplot2}? NB This exercise is primarily intended for those who already have some knowledge of R or who consider themselves proficient programmers.

3. Some Basic R practice

Essential tips for R:

- R is case-sensitive, so price and Price are different names.
- In R everything that exists is an object.
- Functions are also objects: mean refers to the function, while mean(x) calls it.
- `#` in front of a sentence or a word is treated as a comment and is not executed.
- For help in R, `?<function or command>` e.g., `?head`

3.1 A simple variable assignment example

In the previous task you may well have largely relied on the provided R code. Now we move onto learning about basic R syntax and some coding exercises for you to undertake.

Suppose I'm trying to calculate how much money I'm going to make from my *Modern Data* book. There are several different things I will need to store:

- `sales` how many copies I'll sell

- `price` the price per copy
- `numberStudents` number of students in the class (assume I sell one copy per student)
- `recommend` the proportion of students so impressed that they recommend the book to friends and family
- `income` the income generated

Let's turn this into R.

```
# Compute sales income for Martin's book

price <- 10                                 # assume price is 10 UKP per book copy -
                                            # actually it's free but never mind!
numberStudents <- 100                       # number of students in the class
recommend <- 0.5                            # proportion of students who recommend
                                            # the book

sales <- numberStudents * (1 + recommend)
income <- sales * price
paste("Martin will earn:", income)          # display a readable string with the paste function
```

```
## [1] "Martin will earn: 1500"
```

Although we could just output `income`, I have added some text to make the output of the code easier to interpret with the `paste()` function to combine a character string literal delimited by quotation marks and the calculated income.

Exercise 3.1: Change the number of students to 200 and the recommendation rate to 0.2. What's my new estimated income?

Exercise 3.2: Update the sales income R code to handle tax. Assume that income is taxed at a flat rate of 20%. Modify the above R code to output gross income and net (after tax) income. Using RStudio you can edit this Week 1 Worksheet document.

```
# Your extended code needs to go here
```

HINT: I recommend you define a new numeric variable to store the tax rate and another for net income. For clarity you might want to rename `income` to make it obvious that it stores gross income.

Extension Exercise 3.3: How would you answer the question: it's possible to write a much shorter R program without any extra variables, so why bother?

It's very *important* to get into the habit of writing easy to follow R code. For this reason I have added comments and tried to select meaningful variable names.

3.2 Using vectors

Recall that a Vector is a repeated group of the **same** data type, e.g., a vector of character strings.

```
# Define a new vector
greetings <- c("Hi!", "Hello", "Good morning", "Saludo", "Hej", "¡Hola!")
greetings
```

```
## [1] "Hi!"          "Hello"        "Good morning" "Saludo"       "Hej"
## [6] "¡Hola!"
```

We can also check whether the type of `greetings` is a vector.

```
# Check greetings is a vector
is.vector(greetings)
```

```
## [1] TRUE
```

Exercise 3.4: Check the type of the variable `price` (from Section 3.1). What output do you expect?

To fully exploit the power of vectors we can access subsets of elements. To do this we use the square bracket notation `[<n>]` where n is the position of the required vector element.

```
# Different ways to index vector elements
greetings[2]        # Access the 2nd element
```

```
## [1] "Hello"
```

```
n <- 3
greetings[n]        # You can use a variable as an index
```

```
## [1] "Good morning"
```

```
greetings[1:3]      # Or you can specify a range of vector elements
```

```
## [1] "Hi!"          "Hello"        "Good morning"
```

Notice in the above R chunk, that we can access more than one element at a time by using the `:` operator to return a range of elements. The range can be specified by literals as in `[1:3]` or integer[4] variables e.g., `[m:n]`.

[4] If the index variables are not integers R will do its best to coerce the values to integers.

3.3 Using data frames

Data frames are amongst the most useful of the different data structures. These are two dimensional and unlike 2-dimensional matrices can comprise different constituent data types.

If you're not very confident about data frames I suggest you view my short(ish) video here.

Let's start by examining a built-in data set `mtcars` that stores information about motor cars from the 1970s that is stored as a data frame.

```
# Assign the built in data object mtcars to a new data frame df
df <- mtcars

# View the first 6 rows of the data frame df
head(df)
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

Exercise 3.5: How many variables and observations does `df` have? HINT: There are functions `nrow()` and `ncol()` if you can't be bothered to count.

You can use the `$` operator to specify a column/variable from a data frame, for instance:

```
# Display the observations for cylinder numbers from df
df$cyl
```

```
##  [1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8 4 4 4 8 6 8 4
```

```
# Apply the median function
median(df$cyl)
```

```
## [1] 6
```

Exercise 3.6: Using the `$` operator determine the mean and minimum mpg for cars in df.

Exercise 3.7: Update the number of gears for the Mazda RX4 from 4 to 6. HINT: `df$gear` is a vector so you can use an index to reference a particular element.

So we can use the dollar operator to identify columns/variables. What about rows/observations? There are several ways to achieve this. Using indices via the `[ ]` notation notice that since a data frame has two dimensions two arguments are needed, but since we want the entire row we can leave the argument after the comma blank.

```
# To retrieve the entire 2nd row we could have.
df[2,]
```

```
##               mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4 Wag  21   6  160 110  3.9 2.875 17.02  0  1    4    4
```

Exercise 3.8: Modify the above code to retrieve just columns 1-3 of row 2. HINT: remember the : operators from when we were manipulating vectors.

The other way to retrieve specific observations is when the rows have names e.g., "Mazda RX4", or in this case:

```
# Retrieve the observation named "Valiant"
df["Valiant",]
```

```
##          mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Valiant 18.1   6  225 105 2.76 3.46 20.22  1  0    3    1
```

Exercise 3.9: Change the number of cylinders for the Valiant to 12.

Quiz: Please check your understanding of this chapter via this quick quiz (10 simple multiple choice questions with answers).

3.4 Reading in a csv file

So far we have made use of data that is already in R or input our own. In practice often data will be provided in .csv format and R makes it easy to read it in.

- To make your life easier we have hosted the worms file on a GitHub repository.
- Run the code below to read data from the hard coded url.

```
#
worms <- read.csv("https://raw.githubusercontent.com/mjshepperd/CS5702-Data/master/worms.csv")
```

If you want to check what the data looks like:

```
head(worms,n=10)
```

```
##         Field.Name Area Slope Vegetation Soil.pH  Damp Worm.density
## 1      Nashs.Field  3.6    11  Grassland     4.1 FALSE            4
## 2   Silwood.Bottom  5.1     2     Arable     5.2 FALSE            7
## 3    Nursery.Field  2.8     3  Grassland     4.3 FALSE            2
## 4      Rush.Meadow  2.4     5     Meadow     4.9  TRUE            5
## 5  Gunness.Thicket  3.8     0      Scrub     4.2 FALSE            6
## 6         Oak.Mead  3.1     2  Grassland     3.9 FALSE            2
## 7     Church.Field  3.5     3  Grassland     4.2 FALSE            3
## 8          Ashurst  2.1     0     Arable     4.8 FALSE            4
## 9      The.Orchard  1.9     0    Orchard     5.7 FALSE            9
## 10   Rookery.Slope  1.5     4  Grassland     5.0  TRUE            7
```

In QDA you will see how to do some more descriptive data analysis.

Extension Appendix 1: extra notes on YAML

YAML rhymes with camel! It is an acronym for "**Y**AML **A**in't **M**arkup **L**anguage".

An R Markdown file has three components:

1. Header or front matter written in YAML
2. Markdown: this is text to describe your R analysis
3. Code chunks: Chunks of embedded R code that can be executed and rendered

An RMarkdown file *must* start with a header written using YAML syntax. There are four default elements in the RStudio YAML header:

```
title: The title of your document. NB this is not the same as the file name.
author: Who wrote the document.
date: By default this is the date that the file is created.
output: What format will the output be in.  We will generally use html.  Other options are pdf, various
```

The YAML is delineated by `---` and `---`.

However, if you're unsure, then your best strategy is to copy and adapt, e.g., change the date, title or author.

Extension Appendix 2: Pie charts with ggplot2

Remember that many data scientists have strong reservations about pie charts. In this case a bar plot is probably more effective.

NB This optional example will run only if {ggplot2} is installed. If no chart appears, run `install.packages("ggplot2")` in the Console and knit the document again.

```r
# Reproduce the data from the 2025 required tools for Data Scientists
# using ggplot2

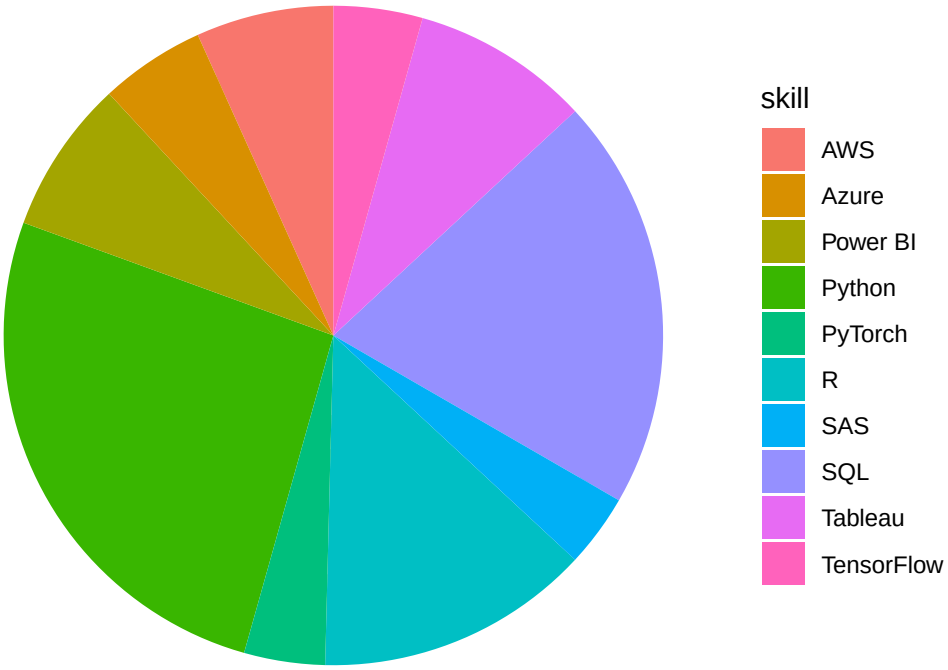
# load the ggplot2 library
library(ggplot2)

skill <- c(
  "Python", "SQL", "R", "Tableau", "Power BI",
  "AWS", "Azure", "TensorFlow", "PyTorch", "SAS"
)
percent <- c(66, 51, 34, 22, 19, 17, 13, 11, 10, 9)

# Merge into a single dataframe which makes ggplot easier
dsTools <- data.frame(skill,percent)

# Generate the pie chart
ggplot(dsTools, aes(x="", y=percent, fill=skill)) +
  geom_bar(stat="identity", width=1) +
  coord_polar("y", start=0) +
  theme_void() +                 # remove background, grid, numeric labels
  labs(title = "In demand data scientist tools (2025)") +
  theme(plot.title = element_text(size = 10))
```

In demand data scientist tools (2025)



Extension Exercise A2 Rewrite the above ggplot2 to produce a bar chart. Add suitable labels.